

Quiz



<https://moodle.gla.ac.uk/mod/quiz/view.php?id=5824753>

Quiz ends and class starts at 14:10



University
of Glasgow



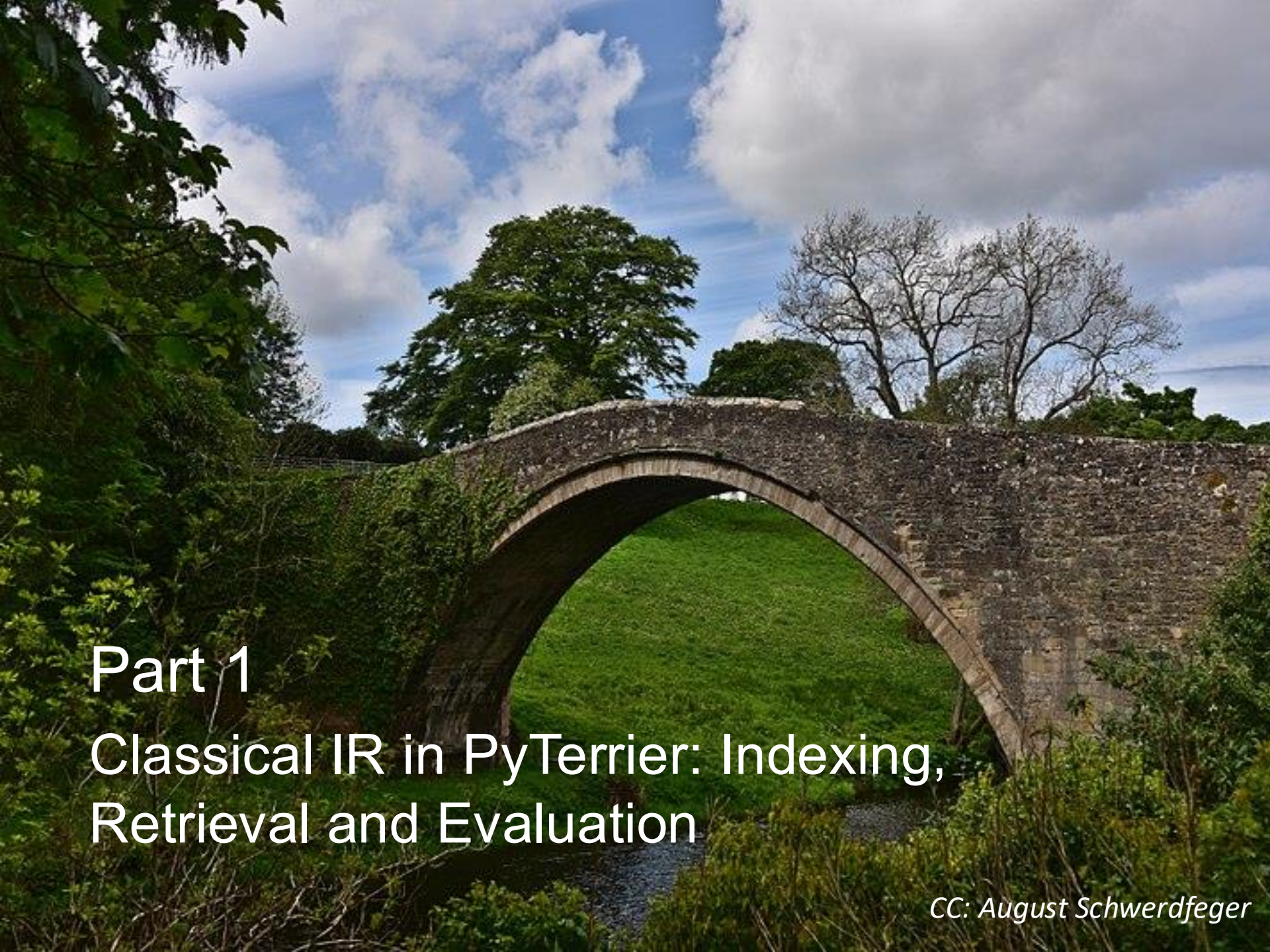
UNIVERSITÀ DI PISA

PyTerrier Tutorial

IR H/M

30th January 2026

Sean MacAvaney

A photograph of a stone arch bridge spanning a river. The bridge is made of dark, rough-hewn stone and features a large, single arch. The river flows beneath the arch, and the surrounding landscape is lush with green trees and foliage. The sky above is blue with scattered white clouds.

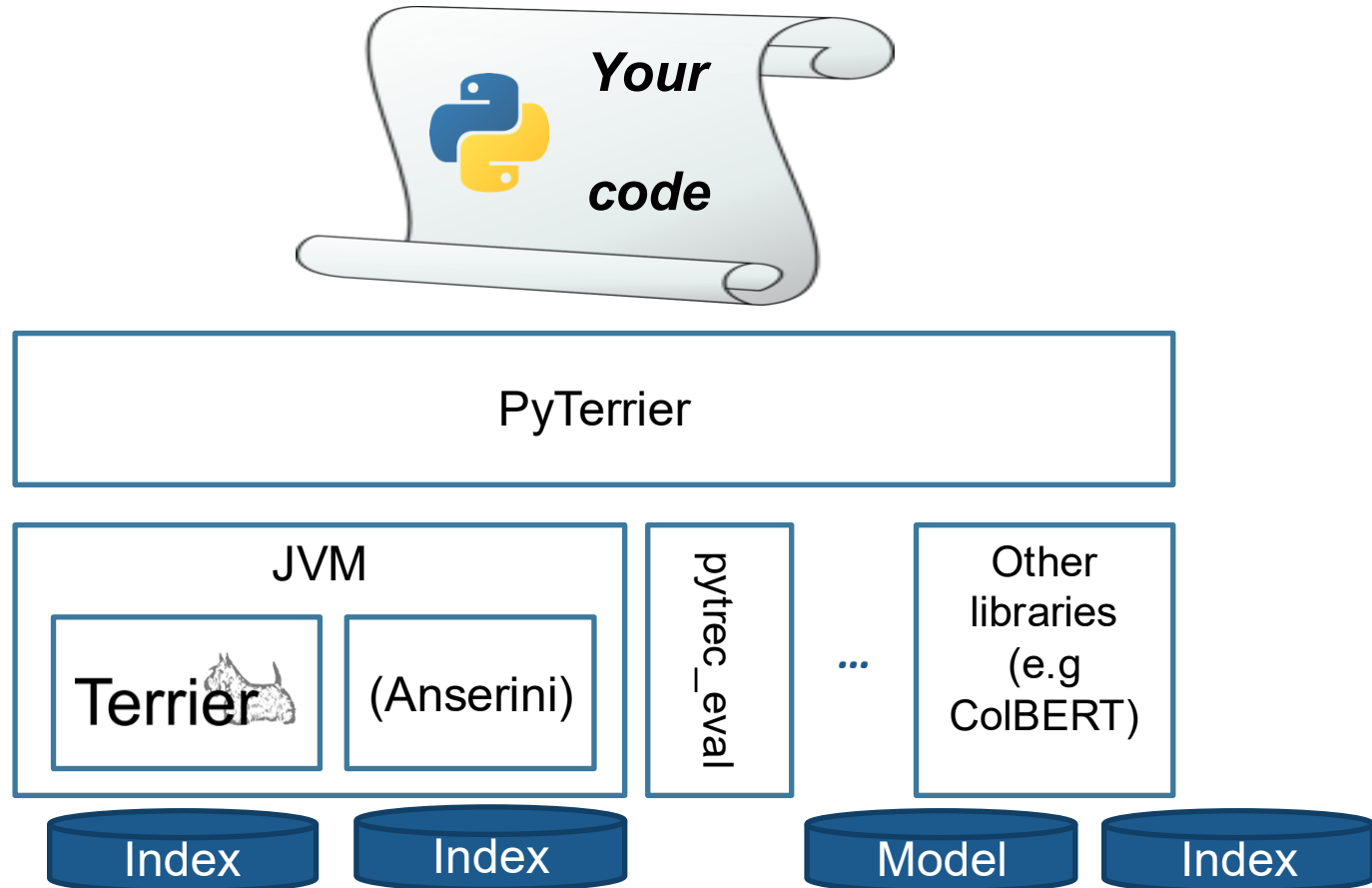
Part 1

Classical IR in PyTerrier: Indexing, Retrieval and Evaluation

What is PyTerrier

A Python layer above Terrier  (and **other** IR platforms)

A Python framework for expressing and evaluating IR experiments



In Part 1

Classical IR in PyTerrier: Indexing, Retrieval and Evaluation

1. Getting started
2. Indexing
3. Retrieval
4. Evaluation

Lets get started...

Install PyTerrier

pip install pyterrier

or

pip install pyterrier[java] (to use Java bindings)

We do semi-regular releases

Get things going: `import pyterrier as pt`

```
terrier-assemblies 5.11 jar-with-dependencies not found, downloading to /root/.pyterri
https://repo1.maven.org/maven2/org/terrier/terrier- 99.6M/99.6M [00:00<00:00, 121MB/
assemblies/5.11/terrier-assemblies-5.11-jar-with- s]
dependencies.jar: 100%
Done
terrier-python-helper 0.0.8 jar not found, downloading to /root/.pyterrier...
https://repo1.maven.org/maven2/org/terrier/terrier-python- 36.6k/36.6k [00:00<00:00, 886kB/
helper/0.0.8/terrier-python-helper-0.0.8.jar: 100% s]
Done
Java started and loaded: pyterrier.java.colab, pyterrier.java, pyterrier.java.24, pyte
```

JAR files for Terrier etc are downloaded automatically

The Format of a (“Sparse”) Index

An index normally contains several data structures

- **Lexicon:** Records the list of all unique terms and their statistics
- **Document Index:** Records the statistics of all documents
- **Inverted Index:** Records the mapping between terms and documents. Contains many posting *lists*.
- **MetaIndex:** Records document metadata
- **Direct Index:** Records terms for each document

Document
“information”
Incl. raw text

Lexicon

term	df	cf	<i>p</i>
------	----	----	----------

DocumentIndex

id	len	<i>p</i>
----	-----	----------

MetaIndex

id	docno	url	...
----	-------	-----	-----

Could also contain other occurrence information:
e.g. term positions, fields (title, URL)

PostingIndex (inverted)

id	tf	id	tf	id	tf
----	----	----	----	----	----

One posting list for each term: each posting represents an occurrence

PostingIndex (direct)

id	tf	id	tf	id	tf
----	----	----	----	----	----

One posting list for each document: each posting represents an occurrence

Creating an Index

Lets consider a few sample documents

```
docs = [  
    {'docno': '1', 'text': 'He ran out of money, so he had to stop playing'},  
    {'docno': '2', 'text': 'The waves were crashing on the shore; it was a'},  
    {'docno': '3', 'text': 'The body may perhaps compensate for the loss'},  
]
```

We can index these documents using TerrierIndex

```
index = pt.terrier.TerrierIndex('pd_index')  
indexer = index.indexer()  
indexer.index(docs)
```

This creates a directory (`pd_index`) with the following files:

data.direct.bf
data.document.fsarrayfile
data.inverted.bf

data.lexicon.fsomapfile
data.lexicon.fsomaphash
data.lexicon.fsomapid

data.meta-0.fsomapfile
data.meta.idx
data.meta.zdata

data.properties

Index Configurations

Record positions to allow proximity/phrase search by adding

store_positions=True **to** indexer()

PostingIndex (inverted)

id	tf	id	tf	id	tf
----	----	----	----	----	----

No positions

Positions

(more space)

PostingIndex (inverted)

id	tf	p	p	p	id	tf	p	id	tf	p	p
----	----	---	---	---	----	----	---	----	----	---	---

Document metadata can be stored in the MetaIndex

- E.g. saving the raw document text can be useful for neural re-rankers
- Minimum/default: “docno” a unique string identifier for each document
- Recording more metadata takes more space and can slow down retrieval

MetaIndex

id	docno	url	text	...
----	-------	-----	------	-----

Accessing Terrier Index Data Structures from Python

Lexicon:

How many documents does 'chemical' appear in?

```
index.lexicon()['chemic'].getDocumentFrequency()
```

DocumentIndex

What is the length of the 23rd document?

```
index.document_index().getDocumentLength(22)
```

Inverted Index

What documents contain 'chemical'?

```
lexicon = index.lexicon()
meta = index.meta_index()
inv = index.inverted_index()

le = lexicon.getLexiconEntry("chemic")
# the lexicon entry is also our pointer to access the inverted index posting list
for posting in inv.getPostings(le):
    docno = meta.getItem("docno", posting.getId())
    print(f"{docno} with frequency {posting.getFrequency()}")
```

You've seen this all in Lab 1

Retrieval

Now we have an index, lets perform retrieval on it. We can access retrievers from a `TerrierIndex` object.

```
ret = index.bm25()
```

We can search it

```
ret.search("chemical reactions")
```

	qid	docid	docno	rank	score	query
0	1	251931	449027_8	0	11.954261	chemical reactions
1	1	251934	449027_11	1	11.091389	chemical reactions
2	1	251926	449027_3	2	11.087365	chemical reactions
3	1	36690	2769813_5	3	10.902714	chemical reactions

Retrieving more than one query

For experiments, often we operate on more than one query. Lets say we have a dataframe of queries, `queryset`

`queryset =`

	<code>qid</code>	<code>query</code>
0	3990512	how can we get concentration on something
1	714612	why doesn't the water fall off earth if it's r...
2	2528767	how do i determine the charge of the iron ion ...

`results = ret.transform(queryset)`

	<code>qid</code>	<code>docid</code>	<code>docno</code>	<code>rank</code>	<code>score</code>	<code>query</code>
0	3990512	173781	4366141_0	0	7.705589e+00	how can we get concentration on something
1	3990512	381364	3378079_2	1	7.485244e+00	how can we get concentration on something
2	3990512	269289	3270641_2	2	7.351503e+00	how can we get concentration on something
...	...	...	...	...	...	...
188628	1340574	291774	2489233_1	997	5.957036e+00	why do some people only go to church on easter...
188629	1340574	259693	3073253_4	998	5.955254e+00	why do some people only go to church on easter...
188630	1340574	265175	3610636_18	999	5.953573e+00	why do some people only go to church on easter...

In fact, the `transform()` method is used so often, it can be omitted

`ret(queryset)`

In the exercises, you will be provided with the topics as dataframes, ready to use, from “datasets”:

`ret(dataset.get_topics())`

Changing the Weighting Model

We expose most useful configuration through the TerrierIndex API. There are many different weighting models (e.g., TF-IDF, Divergence from Randomness, etc.)

```
bm25(*, k1=1.2, b=0.75, num_results=1000, include_fields=None, threads=1, verbose=False) [source]
```

Creates a BM25 retriever for this index.

RETURN TYPE:

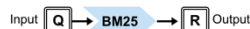
[Transformer](#)

PARAMETERS:

- **k1** (*float*) – BM25's **k1** parameter, which controls TF saturation.
- **b** (*float*) – BM25's **b** parameter, which controls the length penalty.
- **num_results** (*int*) – The maximum number of results to return per query.
- **include_fields** (*List[str] / None*) – The metadata fields to return for each search result.
- **threads** (*int*) – The number of threads to use during retrieval.
- **verbose** (*bool*) – Whether to progress information during retrieval

Example Pipeline:

```
index.bm25()
```



[Citation](#)

- Robertson et al. Okapi at TREC-7: Automatic Ad Hoc, Filtering, VLC and Interactive. TREC 1998.

```
tf_idf(*, k1=1.2, b=0.75, num_results=1000, include_fields=None, threads=1, verbose=False) [source]
```

Creates a TF-IDF retriever for this index.

This retriever uses the Robertson formulation of TF and the Sparck Jones formulation of IDF.

RETURN TYPE:

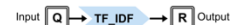
[Transformer](#)

PARAMETERS:

- **k1** (*float*) – TF-IDF's **k1** parameter, which controls TF saturation.
- **b** (*float*) – TF-IDF's **b** parameter, which controls the length penalty.
- **num_results** (*int*) – The maximum number of results to return per query.
- **include_fields** (*List[str] / None*) – The metadata fields to return for each search result.
- **threads** (*int*) – The number of threads to use during retrieval.
- **verbose** (*bool*) – Whether to progress information during retrieval

Example Pipeline:

```
index.tf_idf()
```



[Citation](#)

- Robertson et al. Okapi at TREC-7: Automatic Ad Hoc, Filtering, VLC and Interactive. TREC 1998.

[Citation](#)

- Jones. A statistical interpretation of term specificity and its application in retrieval. J. Documentation 2004. [\[link\]](#)

<https://pyterrier.readthedocs.io/en/latest/terrier/api.html>

More information about these models in Lecture 7

Do we even need an index?

Imagine our dataframe contains query and doc texts

```
index.bm25(include_fields=["docno", "text"])
```

	qid	query	docno	text
0	q1	chemical reactions	d1	professor proton poured the chemicals
1	q1	chemical reactions	d2	chemical brothers turned up the beats

We can score that directly using a weighting model

```
tfs = pt.text.scorer(wmodel="Tf")
```

```
tfs.transform(query_docs)
```

	qid	docno	rank	score	query
0	q1	d1	0	1.0	chemical reactions
1	q1	d2	1	1.0	chemical reactions

```
BM25s = pt.text.scorer(wmodel="BM25", background_index=index)
```

```
BM25s.transform(query_docs)
```

	qid	docno	rank	score	query
0	q1	d1	0	13.823546	chemical reactions
1	q1	d2	1	13.823546	chemical reactions



These are like re-ranking operations – we'll talk more about these later in the course

Evaluating Pipelines

A basic IR experiment typically has 3 procedural steps:

- obtain the queries Q and the corresponding relevance assessments (qrels) RA , corresponding to the documents in the index
- transform those queries into results using the retriever instance, let's say

$$R = \text{retrieve}(Q)$$

- apply an evaluation tool, such as `trec_eval` on RA and R (saved as files?), to obtain effectiveness measures such as MAP or NDCG

This would need to be repeated for each retrieval instance

Also complex to compute things like number of queries improved, significance testing, etc.

An Experiment Abstraction for Evaluation

For evaluation, we need queries Q and the corresponding relevance assessments (qrels) RA , corresponding to the documents in the index

PyTerrier defines an **Experiment abstraction**, which performs an evaluation of multiple systems on queries Q and labels RA

```
pt.Experiment([ret1, ret2], Q, RA, [MAP, P@1])
```

Returns a dataframe with measure values for each system

This is **declarative** in nature – we say what we want, not how to get it

- No writing files and then evaluating
- No dealing with results. Experiment does this for you – we have abstracted away from the results entirely

Experiment Example

An example Experiment might look like

```
from pyterrier.measures import *
pt.Experiment(
    [tfidf, bm25],
    vaswani.get_topics(),
    vaswani.get_qrels(),
    [MAP, NDCG@10])
```

- ← Import the available measures
- ← What systems are we evaluating?
- ← With what topics?
- ← What qrels for those topics?
- ← Name the measure, @ cutoff

This outputs a dataframe as follows:

	name	AP	nDCG@10
0	TerrierRetr(TF_IDF)	0.290905	0.444411
1	TerrierRetr(BM25)	0.296517	0.446609

*This is due to **ir_measures** – an elegant evaluation tool by Sean. It wraps (py)trec_eval, ndeval, cwl_eval, etc..*

Experiment Example, with names

We can put names in the output dataframe

```
from pyterrier.measures import *
pt.Experiment(
    [tfidf, bm25],
    vaswani.get_topics(),
    vaswani.get_qrels(),
    [MAP, NDCG@10],
    names=["TF_IDF", "BM25"])
```

Names of our approaches

Resulting output:

	name	AP	nDCG@10
0	TF.IDF	0.290905	0.444411
1	BM25	0.296517	0.446609

Experiments Variations (1)

Scientifically sound conclusions needs a baseline

- And **significance testing** [see last part of Lecture 5]

Declaring a baseline – specify the system you want to compare to

```
pt.Experiment(
    [tfidf,bm25],
    vaswani.get_topics(),
    vaswani.get_qrels(),
    [MAP, NDCG@10],
    baseline=0,
    names=["TF_IDF", "BM25"])
```

Index of the baseline system

	name	AP	nDCG@10	AP +	AP -	AP p-value	nDCG@10 +	nDCG@10 -	nDCG@10 p-value
0	TF_IDF	0.290905	0.444411	NaN	NaN	NaN	NaN	NaN	NaN
1	BM25	0.296517	0.446609	46.0	45.0	0.237317	16.0	23.0	0.63001

of queries improved
and degraded

P-value of two-tailed
paired t-test

Repeated for each
measure

Experiment Variations (2)

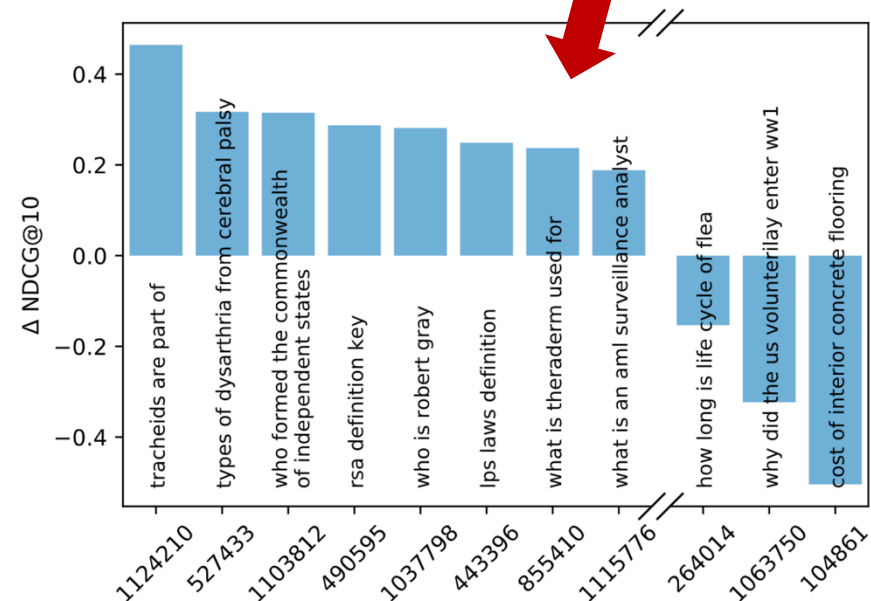
Which queries were affected

```
pt.Experiment(  
    [tfidf, bm25],  
    vaswani.get_topics(),  
    vaswani.get_qrels(),  
    [MAP, NDCG@10],  
    perquery=True)
```

How do we get this?

Breakdown per query

	name	qid	measure	value
0	TF.IDF	1	map	0.268860
1	TF.IDF	1	ndcg_cut_10	0.573690
2	TF.IDF	2	map	0.056448
3	TF.IDF	2	ndcg_cut_10	0.094788



This will be important in Exercise 1

Round Up

We have **reviewed** the classical IR infrastructure, as implemented by PyTerrier, namely:

- Indexing, including accessing an index and datasets
- Retrieval & Evaluation

Coming Next: You have seen several retrieval objects: `index.bm25()` and `pt.text.scorer()`. **These are two transformers**

- They *transform* a dataframe into another dataframe
- Next, we'll generalise what we have seen into different types of transformers, and show how to combine them



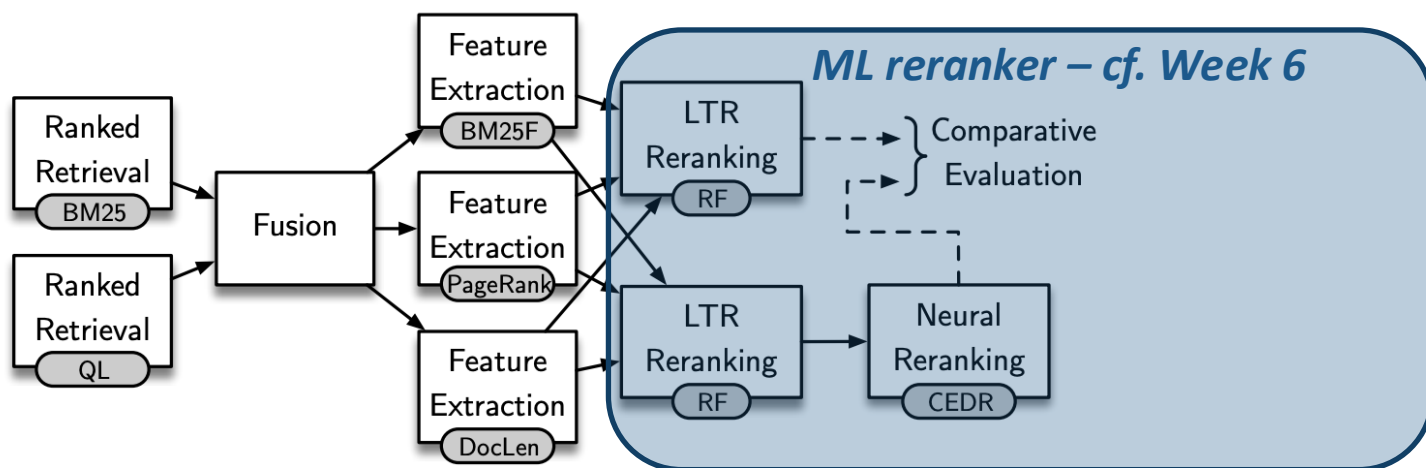
Part 2: PyTerrier data model and transformers

Bridging to Modern Retrieval Architectures

We have seen how to perform an experiment in a declarative manner, and perform simple retrievals

Actually, in IR, we have many retrieval and re-ranking components

Working with and combining result files to allow comparing complex ranking *pipelines* is tricky:



In this part, we **generalise** the architecture that has been seen into the PyTerrier data model and operators, to make it easy to build complex ranking pipelines

PyTerrier makes it easy to construct complex IR pipelines

Key components:

➡ **A common data model**

- A rich library of transformation functions

including classical retrieval, query reformulation, learning-to-rank, neural re-ranking

- Operators to combine transformers

You will learn about all of these in the IR course

Common Data Model



Q

<u>qid</u>	query
0	gold coast temperature november
1	one way flight to gold coast price
...	...



D

<u>docno</u>	text	title
0	Science & Mathematics Physi...	The hot glowing...
1	School-Age Kids Growth &...	Developmental...
...	...	...



R

<u>qid</u>	query	<u>docno</u>	score
0	gold coast temperature november	15213	0.5134
0	gold coast temperature november	42635	0.3742
0	gold coast temperature november	26340	0.3223
...	...	...	...

$$R \subset Q \times D$$



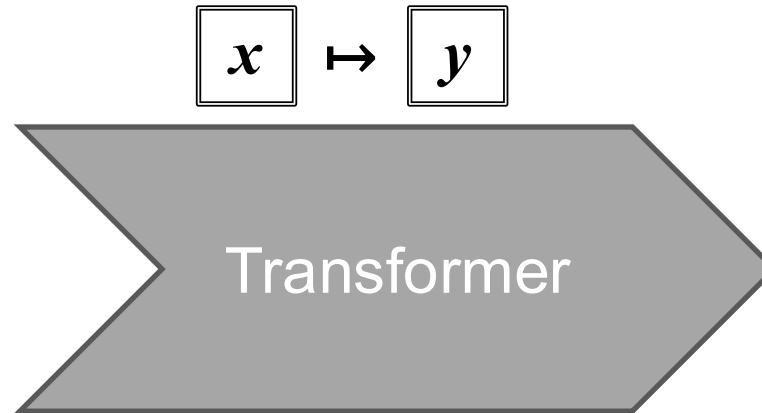
RA

<u>qid</u>	<u>docno</u>	label
0	15213	1
...	...	...

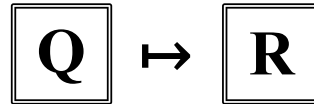
$$RA \subset Q \times D$$

(All of these are Pandas dataframes, i.e. relations)

Transformation Function Objects (Transformers)



Transformation Function Objects (Transformers)



qid	query
0	gold coast temperature...
1	one way flight to gold...
...	...

qid	query	docno	score
0	gold coast...	15213	0.5134
0	gold coast...	42635	0.3742
0	gold coast...	26340	0.3223
...	...	...	...

Transformation Function Objects (Transformers)

$$\boxed{Q} \mapsto \boxed{R}$$

Your latest
retrieval
approach...

qid	query
0	gold coast temperature...
1	one way flight to gold...
...	...

qid	query	docno	score
0	gold coast...	43242	0.1252
0	gold coast...	13746	0.1196
0	gold coast...	86454	0.0934
...	...	...	...

Transformation Function Objects (Transformers)

Transformer Class	Examples
$\boxed{Q} \mapsto \boxed{R}$ Retrieval	TF.IDF, BM25 (Lect. 7), Dense retrieval (Lect. 12)

Transformation Function Objects (Transformers)

Transformer Class	Examples
$\boxed{Q} \mapsto \boxed{R}$ Retrieval	TF.IDF, BM25 (Lect. 7), Dense retrieval (Lect. 12)
$\boxed{R} \mapsto \boxed{Q}$ Query Expansion	RM3, Bo1, etc. (Lect. 6)

Uses the top-retrieved documents to expand and refine the query automatically – coming next week in Lecture 6

Transformation Function Objects (Transformers)

Transformer Class	Examples
$\boxed{Q} \mapsto \boxed{R}$ Retrieval	TF.IDF, BM25 (Lect. 7), Dense retrieval (Lect. 12)
$\boxed{R} \mapsto \boxed{Q}$ Query Expansion	RM3, Bo1, etc.
$\boxed{R} \mapsto \boxed{R}$ Re-ranking	Neural reranking (e.g. using BERT; Lect. 12)
$\boxed{Q} \mapsto \boxed{Q}$ Query Re-writing	Sequential Dependence model (Lect. 8)
$\boxed{Q} \mapsto \boxed{A}$ Question Answering	LLMs like FlanT5, LLama
$\boxed{R} \mapsto \boxed{A}$ Retrieval Augmented Generation QA	Fusion-in-Decoder

NEW

```
[12] index = pt.IndexRef.of("./cord19-index")
      pt.terrier.Retriever(index, wmodel='B2M25')

      bm25(topics)
```

	qid	docid	docno	rank	score	query
0	1	83032	dv9m19yk	0	11.865104	what is the origin of covid 19
1	1	109719	kgifmjyb	1	11.412908	what is the origin of covid 19
2	1	135553	wmfcey6f	2	11.397281	what is the origin of covid 19
3	1	68472	4dtk1kyh	3	10.805808	what is the origin of covid 19
4	1	114244	cniyembt	4	10.763978	what is the origin of covid 19
...	...	...	...	...	...	...
49995	50	146581	2o8u4eny	995	17.073736	what is known about an mrna vaccine for the sa...
49996	50	76091	fayr38c7	996	17.071367	what is known about an mrna vaccine for the sa...
49997	50	102986	jlrk8fg0	997	17.070633	what is known about an mrna vaccine for the sa...
49998	50	183502	imwa033f	998	17.070633	what is known about an mrna vaccine for the sa...
49999	50	95511	j5j3hxxw	999	17.070486	what is known about an mrna vaccine for the sa...

BM25 Results


```
[22] rm3 = pt.terrier.rewrite.RM3(index)
      rm3(bm25(topics))
```

	qid	query	query_0
0	1	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
1	10	19^0.085714296 across^0.019867204 epidem^0.029...	has social distancing had an impact on slowing...
2	11	walk^0.030252097 urolog^0.042577028 optim^0.04...	what are the guidelines for triaging patients ...
3	12	older^0.041744966 best^0.100000009 quarantin^0...	what are best practices in hospitals and at ho...
4	13	contact^0.026820807 possibl^0.021502888 outbre...	what are the transmission routes of coronavirus
5	14	centuri^0.020000001 manag^0.020000001 contain^...	what evidence is there related to covid 19 sup...
6	15	bodi^0.167176828 can^0.120000005 cold^0.034101...	how long can the coronavirus live outside the ...
7	16	stabl^0.152924821 dai^0.042988449 surfac^0.218...	how long does coronavirus remain stable on sur...

Expanded query

Original query

Q

```
[23] bm25(rm3(bm25(topics)))
```

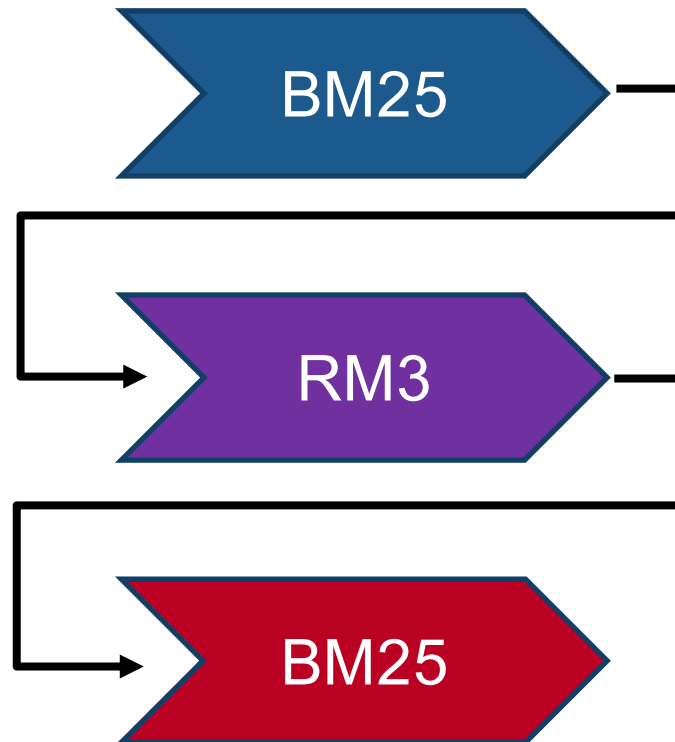
	qid	docid	docno	rank	score	query	query_0
0	1	109719	kgifmjvb	0	19.316403	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
1	1	135553	wmfcey6f	1	19.281917	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
2	1	83032	dv9m19yk	2	11.101417	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
3	1	103317	ec8lpgl3	3	11.044733	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
4	1	114244	cniyembt	4	10.865901	19^0.200000018 origin^0.325867474 covid^0.2000...	what is the origin of covid 19
...	...	...	...	...	...	...	...
49995	9	164232	6djfpvz7	995	6.987903	19^0.150000006 covid^0.150000006 mp^0.05102380...	how has covid 19 affected canada
49996	9	68116	6k8vedy3	996	6.983706	19^0.150000006 covid^0.150000006 mp^0.05102380...	how has covid 19 affected canada
49997	9	132455	9z0azq8y	997	6.983706	19^0.150000006 covid^0.150000006 mp^0.05102380...	how has covid 19 affected canada
49998	9	132456	wfss2j7v	998	6.983706	19^0.150000006 covid^0.150000006 mp^0.05102380...	how has covid 19 affected canada
49999	9	163423	n76892ur	999	6.981468	19^0.150000006 covid^0.150000006 mp^0.05102380...	how has covid 19 affected canada

Results of BM25 over RM3

R

Operators

```
bm25 (rm3 ( bm25 (topics)) )
```

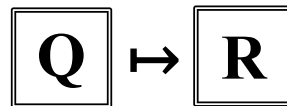
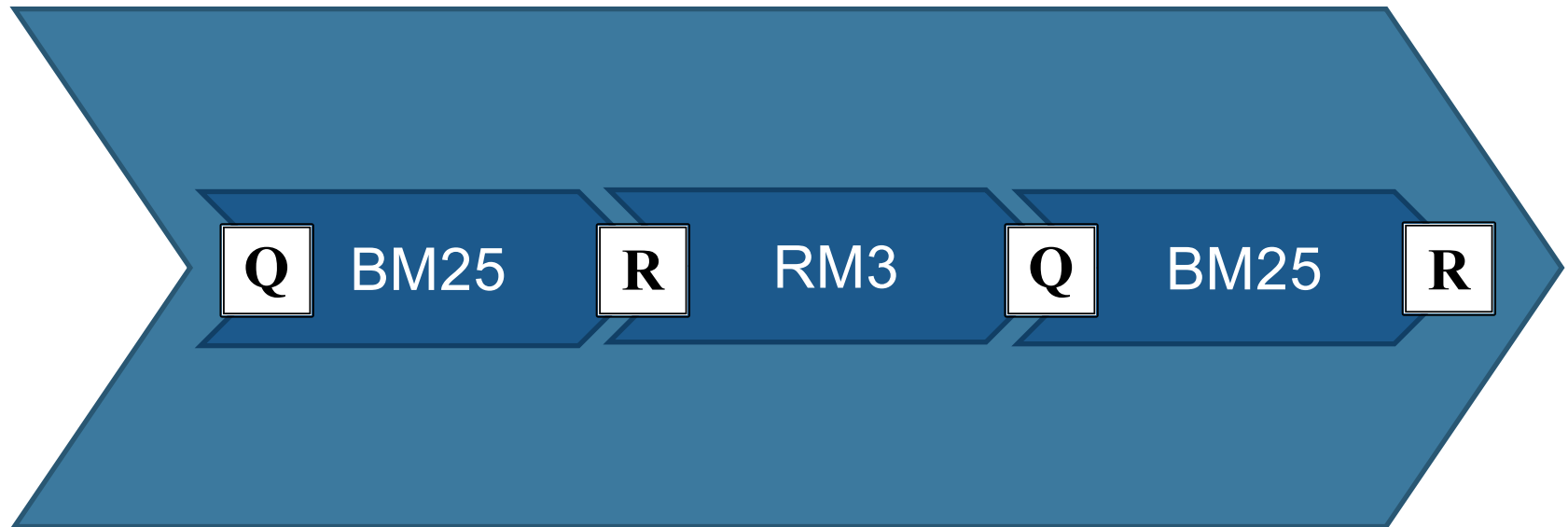


Operators

A better way: the “then” operator

$A \gg B$ (A then B)

pipeline = bm25 $\gg$ rm3 $\gg$ bm25



Operators

A better way: the “then” operator

`A >> B (A then B)`

```
index.bm25() >> index.rm3() >> index.bm25()
```



*PyTerrier generates these schematics
automatically from notebook cell outputs!*

Transformer Operators

PyTerrier defines a suite of operators on transformers

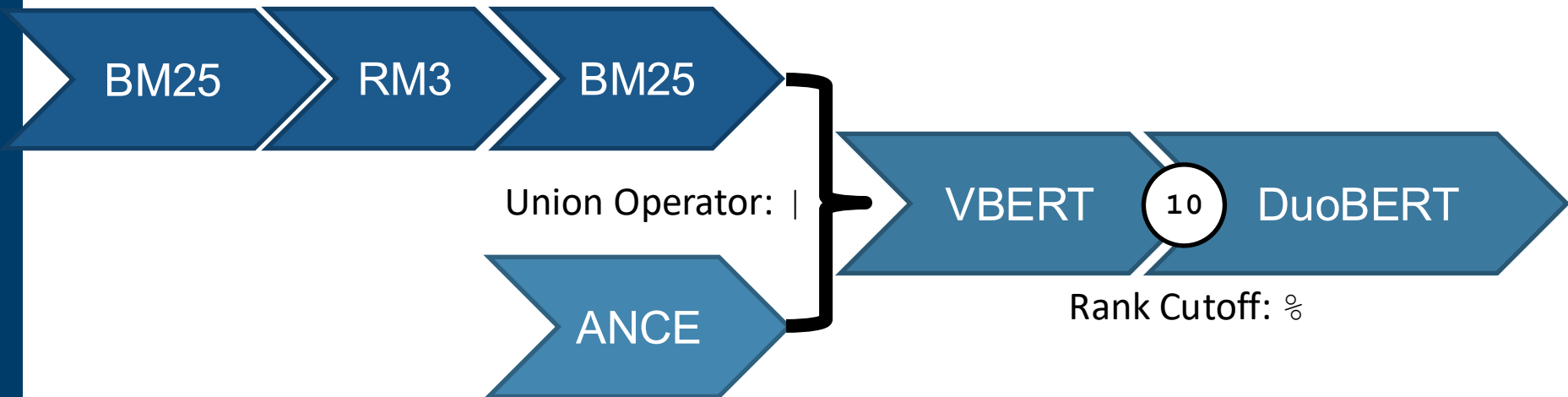
Op.	Name	Description
>>	<i>then (composition)</i>	Pass the output from one transformer to the next transformer
+	<i>linear combine</i>	Sum the query-document scores of the two retrieved results lists
*	<i>scalar product</i>	Multiply the query-document scores of a retrieved results list by a scalar
**	<i>feature union</i>	Combine two retrieved results lists as features
	<i>set union</i>	Make the set union of documents from the two retrieved results lists
&	<i>set intersection</i>	Make the set intersection of the two retrieved results lists
%	<i>rank cutoff</i>	Shorten a retrieved results list to the first K elements
^	<i>concatenate</i>	Add the retrieved results list from one transformer to the bottom of the other

Important
in Ex2

Each PyTerrier operator is defined on dataframes (relations) and has relational algebra semantics

Operators

pipeline = `bm25 >> rm3 >> bm25`



Other operators available, including caching, linear score combination, etc.

Exercise

Let's say that we have two retrievers, for TF.IDF and BM25. How could I formulate a pipeline that ranks documents based on multiplying the BM25 score by three and summed with the TF_IDF score multiplied by two

`index.tf_idf()`

`index.bm25()`

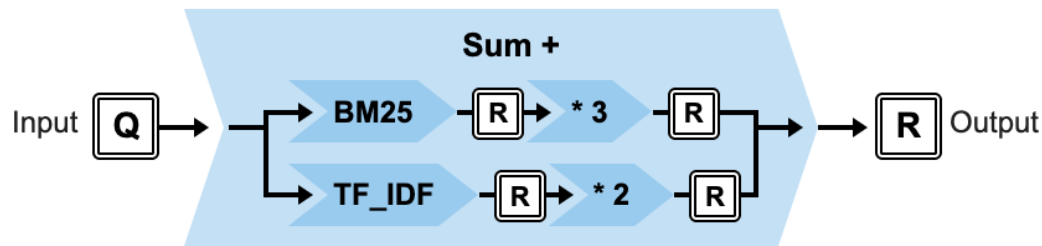
Obtain the scores for this on a topic set called queries

What datamodel types are involved in this transformation?

Answer

Linear Combinations

`linear = index.bm25() * 3 + index.tf_idf() * 2`



`results = linear(queries)`

or

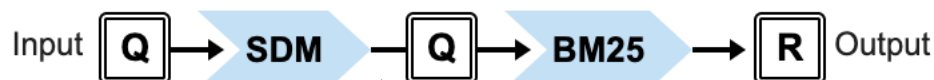
`results = linear.transform(queries)`

(All are R dataframes)

More Examples

1. Sequential Dependence Model (proximity)

```
pt.rewrite.SequentialDependence() >> index.bm25()
```



qid		query
0	1	#combine:0=0.1:wmodel=org.terrier.matching.models.dependence.pBiL(#1(chemical reactions)) #combine:0=0.1:wmodel=org.terrier.matching.models.dependence.pBiL(#uw8(chemical reactions)) #combine:0=0.1:wmodel=org.terrier.matching.models.dependence.pBiL(#uw12(chemical reactions))

2. Query Expansion (aka PRF)

```
index.bm25() >> index.rm3() >> index.bm25()
```



Transformer Implementations & Rolling your Own

All transformers implement a base class called `pt.Transformer`

- This has all the methods to support operator overloading, e.g. `__add__()`, `__rshift__()`

However, creating your own transformer is easy using `pt.apply` functions. These create a new transformer from your own function

- For instance, to rewrite a query – `pt.apply.query()`

```
# this will remove pre-defined stopwords from the query
stops = set(["and", "the"])

# a naive function to remove stopwords – takes as input a Pandas Series, and returns
def _remove_stops(q):
    terms = q["query"].split(" ")
    terms = [t for t in terms if not t in stops]
    return " ".join(terms)

# a query rewriting transformer that applies the _remove_stops to each row of an input
p1 = pt.apply.query(_remove_stops) >> index.dph()
```

- `pt.apply.query()` returns a transformer, which applies `_remove_stops` to each query. The previous query is saved in the `query_0` attribute

There are other `pt.apply` functions – some will be needed in Ex 2

Transformer Implementations (2/2): *pt.apply.doc_score()*

***pt.apply.doc_score()* uses a function that returns a score**

```
# this transformer will subtract 5 from the score of each document
p = pt.terrier.Retriever(index, wmodel="DPH") >>
  pt.apply.doc_score(lambda doc : doc["score"] -5)
```

In this way, ANY function that can returns a number can be used for retrieval

- i.e. any (SOTA) approach that can be expressed as a function of the **text of the document** and the **text of the query** can easily be used as a re-ranker in PyTerrier

***pt.apply.doc_score()* & *pt.apply.query()* have known semantics**

- There are also advanced “generic” apply functions that can apply an arbitrary change of a dataframe as a transformer

Summary: PyTerrier Transformer Operators

Op.	Name	Description
>>	<i>then</i> (<i>composition</i>)	Pass the output from one transformer to the next transformer
+	<i>linear combine</i>	Sum the query-document scores of the two retrieved results lists
*	<i>scalar product</i>	Multiply the query-document scores of a retrieved results list by a scalar
**	<i>feature union</i>	Combine two retrieved results lists as features (Exercise 2)
	<i>set union</i>	Make the set union of documents from the two retrieved results lists
&	<i>set intersection</i>	Make the set intersection of the two retrieved results lists
%	<i>rank cutoff</i>	Shorten a retrieved results list to the first K elements
^	<i>concatenate</i>	Add the retrieved results list from one transformer to the bottom of the other

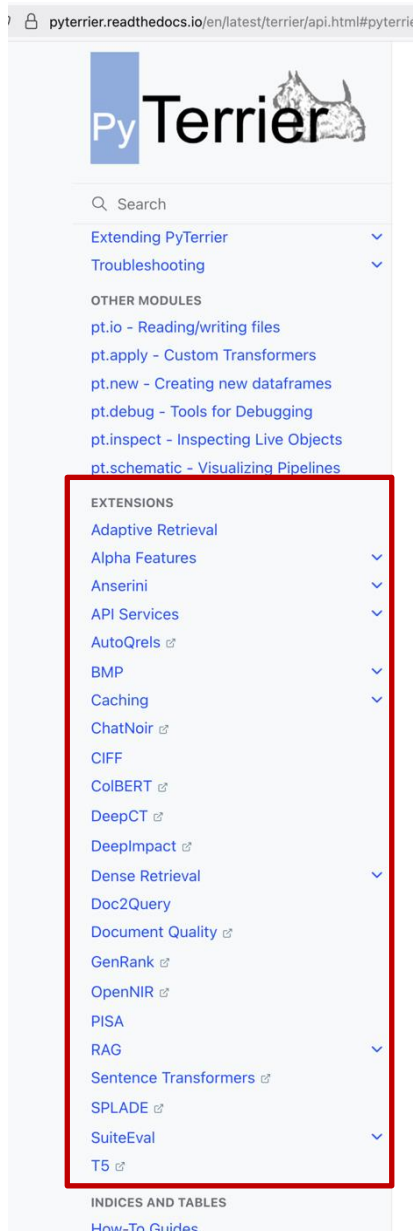
Some final thoughts

The declarative manner of PyTerrier is well-suited to Python, and conducting complex experiments in information retrieval

The Transformers and the operators allow you to declaratively build and evaluate pipelines, without needing to fully grasp the underlying implementations

Imperative programming is more of a hinderance. Avoid it at all costs in your Exercises

PyTerrier Ecosystem



PyTerrier is a core platform for making retrieval pipelines and performing retrieval experiments... the value is in the ecosystem

Large ecosystem of extension packages

Easy to make a state-of-the-art retrieval pipeline for many information seeking tasks

```
monoT5 = MonoT5ReRanker(batch_size=64, verbose=False)
fid = pyterrier_rag.readers.T5FiD("terrierteam/t5fid_base_nq")

rag_pipe = bm25_ret % 100 >> monoT5 %10 >> fid

pt.Experiment(
    [rag_pipe],
    dataset.get_topics('dev'),
    dataset.get_answers('dev'),
    [pyterrier_rag.measures.F1, pyterrier_rag.measures.EM],
)
```

Example: Retrieval-Augmented Generation (RAG)

Conclusions

We walked through the main functionalities of PyTerrier needed upto & including Exercise 1

If you study this tutorial and complete the Warmup Exercise, you should be in good shape to undertake Exercise 1, while embracing the spirit of declarative programming

Over the next few weeks you will get more of the theory behind some of the techniques touched upon here, e.g. BM25, query expansion, machine learning in IR

I'll post more questions about PyTerrier in a Moodle quiz next week